Name:
Roll no.:
Class:

1)Introduction

This project is a **Dynamic Financial Product Discovery Platform** developed using React. Unlike traditional e-commerce systems, this application is designed specifically for financial decision-making, where users explore products based on **risk, return, liquidity, and investment goals**.

The system allows users to:

- Browse financial products such as savings, investments, insurance, and crypto assets
- Apply multi criteria filters
- Create a personalized financial profile
- Receive dynamic recommendations based on financial logic
- Build and manage an investment portfolio

The core objective of this project is to **simulate real world fintech platforms**, where user decisions are guided by **financial reasoning rather than static product listings**.

2)Component hierarchy diagram

Overall Structure

The application follows a modular and reusable component-based architecture. The structure is divided into:

- **Pages (Route-level components)**
- **Reusable UI Components**
- **Context Providers (Global State)**
- **Utility Functions (Logic layer)**

```
App
├── Navbar
├── Routes
│   ├── Home
│   ├── ProductListing
│   │   ├── FilterPanel
│   │   └── ProductList
│   │       └── ProductCard
│   ├── ProductDetail
│   ├── ProfilePage
│   │   └── ProfileForm
│   ├── PortfolioPage
│   │   ├── PortfolioSummary
│   │   └── PortfolioItem
│   └── RecommendationsPage
│       └── RecommendationList
├── PortfolioContext Provider
└── UserProfileContext Provider
```

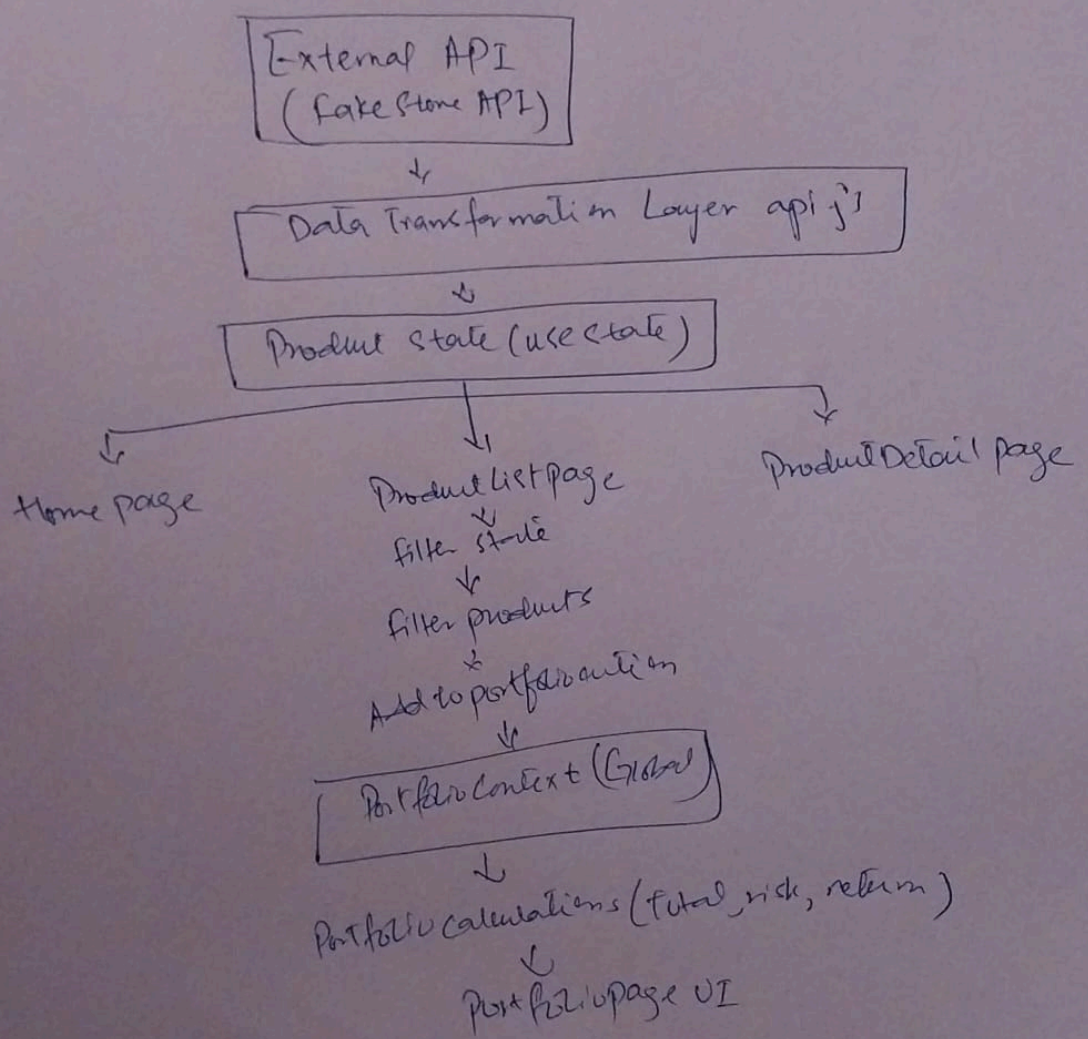
Component Responsibilities

- **ProductCard**: Displays product summary and handles “Add to Portfolio”
- **ProductList**: Renders filtered products
- **FilterPanel**: Manages all filtering inputs
- **ProductDetail**: Shows complete financial data + decision insights
- **ProfileForm**: Collects and updates user financial profile
- **PortfolioSummary**: Displays portfolio analytics
- **RecommendationList**: Displays recommended products

2)State Management

Name:- Syeda Anusha Akbar
Roll no.:- 231-5504

:- State Flow Diagram:-



User Input Flow USER → Profile form → User Profile Context
↓
Recommendation Engine
↓
Recommendation page ← Filtered Recommended Products

Local State (useState)

Local state is used for:

- Form inputs (ProfileForm)
- Filter values (FilterPanel)
- UI states (loading, error)

Example:

```
const [filters, setFilters] = useState({

 risk: [],

 category: [],

 minReturn: 0,

 maxReturn: 30

});
```

Global State (Context API)

Two main contexts are used:

A. PortfolioContext

Manages:

- Portfolio items
- Total investment
- Weighted return
- Risk distribution

Key functions:

- `addToPortfolio(product, amount)`
- `removeFromPortfolio(id)`
- `updateAllocation(id, amount)`
- `calculatePortfolioStats()`

B. UserProfileContext

Manages:

- User financial profile
- Recommendation logic

Key functions:

- `updateProfile(profile)`
- `getRecommendations(products)`
- `isProfileComplete()`

Why Context API?

Using Context avoids **prop drilling**, especially for:

- Portfolio (used across many pages)
- User profile (affects recommendations globally)

State Flow

1. User updates profile
2. Profile state updates in context
3. Recommendation function recalculates
4. UI updates automatically

4. Financial Logic

This is the most important part of the project.

Financial Data Model

Each product includes:

- Expected Return (%)
- Risk Level (low/medium/high)
- Liquidity (easy/moderate/locked)
- Time Horizon (short/medium/long)
- Minimum Investment

Risk-Return Mapping

The system ensures **real-world consistency**:

Risk Level	Return Range
Low	3–7%

Medium	7–12%
High	12–25%

This ensures:

- No unrealistic combinations
- Logical financial behavior

Recommendation Engine

The recommendation engine filters products based on user profile:

Step 1: Risk Filtering

conservative -> low

moderate ->low, medium

aggressive -> all

Step 2: Time Horizon Filtering

short -> short

medium -> short, medium

long -> all

Step 3: Liquidity Filtering

easy -> easy

moderate -> easy, moderate

locked -> all

Step 4: Budget Filtering

product.minInvestment <= user.monthlyCapacity

Step 5: Final Filtering (AND Logic)

All conditions must be satisfied simultaneously.

Step 6: Sorting Logic

- Conservative -> lowest risk first
- Aggressive -> highest return first

Decision Insight Generation

Dynamic insights are generated:

Example:

- Low risk -> “Suitable for conservative investors”
- Locked -> “Requires long-term commitment”
- Long horizon -> “Best for long-term growth”

This makes the system **explainable**, not just functional.

Portfolio Calculations

1. Total Investment

Total = sum of all allocations

2. Weighted Return

Weighted Return = Σ (allocation / total) × product.return

3. Risk Distribution

% Low Risk = (low allocation / total) × 100

4. Category Distribution

Breakdown across:

- Savings
- Investment
- Crypto
- Insurance

Portfolio Constraints

- Prevents unrealistic allocation
- Encourages diversification

5)API Integration and Data Transformation

API Used

Fake Store API is used as the base data source.

Transformation Logic

Since API is not financial, it is transformed into financial products.

Category Mapping

electronics → investment

jewelery → savings

men clothing → insurance

women clothing → crypto

Risk Assignment

investment → medium

savings → low

insurance → low

crypto → high

Return Assignment

Based on risk:

- Low -> small stable returns
- High -> volatile high returns

Other Attributes

- Liquidity -> based on category
- Time horizon -> based on risk
- Min investment -> derived from price

5.3 Why Transformation is Important

- Converts generic data into financial domain
- Ensures consistency
- Makes recommendation system meaningful

6. Challenges and Solutions

Challenge 1: Combining Multiple Filters

Problem: Managing multiple filters together
Solution: Used AND logic so product must satisfy all conditions

Challenge 2: Financial Consistency

Problem: Avoid unrealistic values
Solution: Created strict risk-return mapping

Challenge 3: State Management

Problem: Data needed across multiple pages
Solution: Implemented Context API

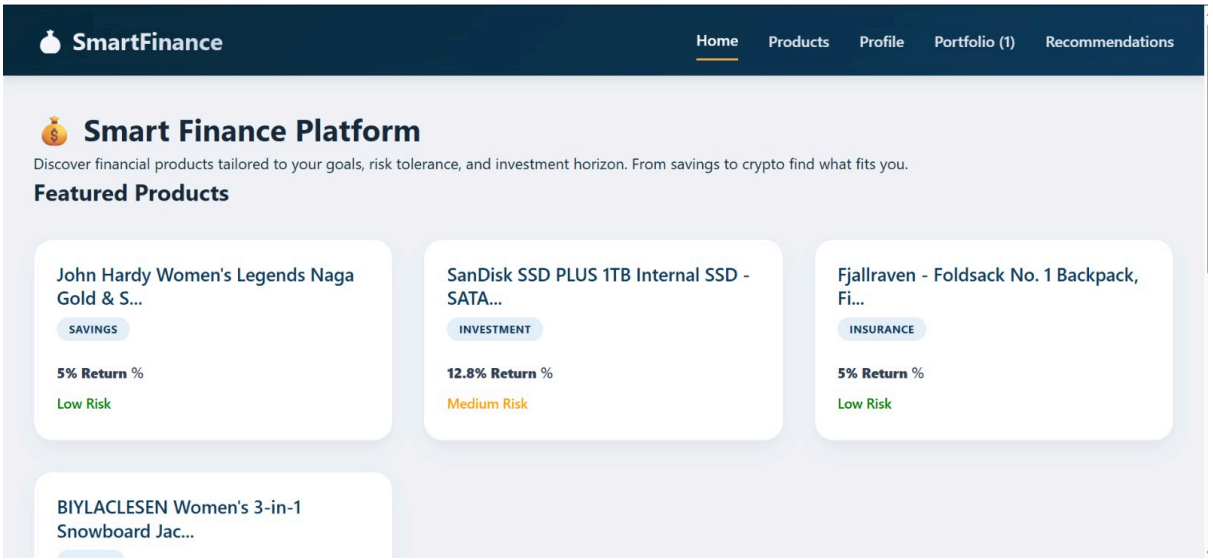
Challenge 4: Recommendation Logic

Problem: Making it dynamic
Solution: Built rule-based filtering system

Challenge 5: Portfolio Calculations

Problem: Calculating weighted return
Solution: Used proportional formulaState Management

7)ScreenShots



SmartFinance

Home | Products | Profile | Portfolio (1) | Recommendations

All Products

Filters

Risk Level

- low
- medium
- high

Category

- savings
- investment
- insurance
- crypto

Liquidity

All

Fjallraven - Foldsack No. 1 Backpack, Fi...

INSURANCE

5% Return %

Risk: Low

Liquidity: locked

Min Investment: PKR 219,900

View Details | Add to Portfolio

Mens Casual Premium Slim Fit T-Shirts

SmartFinance

Home | Products | Profile | Portfolio (1) | Recommendations

My Financial Profile

Risk Tolerance

Conservative (Low risk)

Investment Horizon

Short (1-2 years)

Monthly Investment Capacity (PKR)

29999

Liquidity Preference

Need quick access

Investment Goal

SmartFinance

Home | Products | Profile | Portfolio (1) | Recommendations

My Portfolio

Mens Casual Premium Slim Fit T-Shirts

Category: insurance

Return: 5%

Risk: low

Allocated Amount (PKR): 44600 Update

Remove

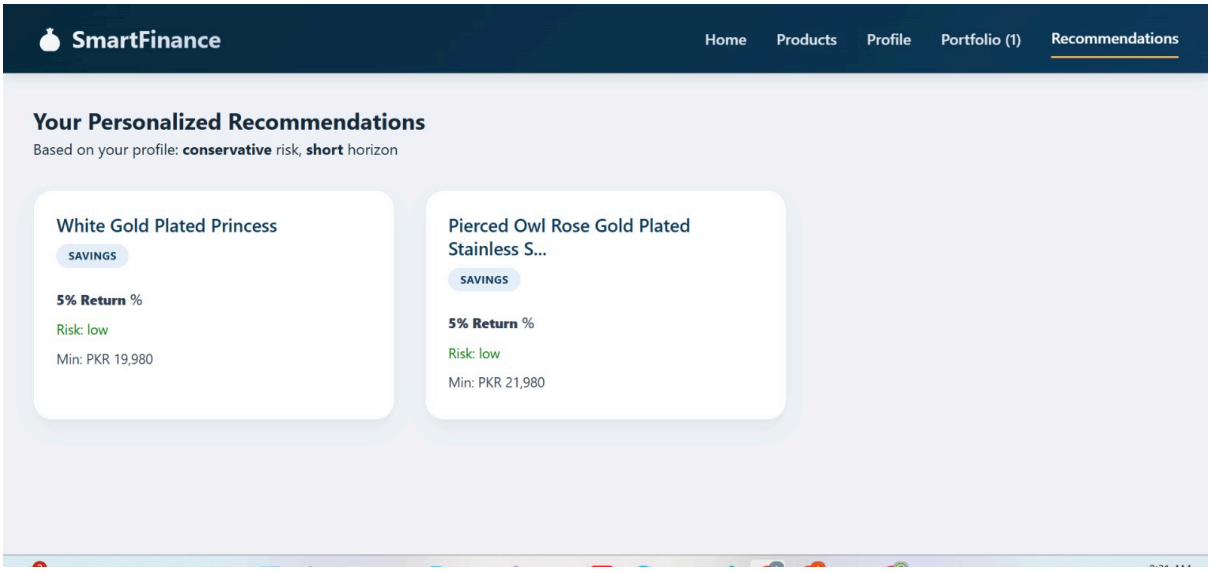
Portfolio Summary

Total Invested: PKR 44,600

Weighted Expected Return: 5%

Risk Distribution

Low Risk:



Workflow

Step 1:

System fetches product data from API

Step 2:

Transforms into financial attributes:

- risk
- return
- liquidity
- investment

Step 3:

User creates profile

Step 4:

System filters products using:

- risk mapping
- horizon mapping
- liquidity mapping
- capacity constraint

Step 5:

Filtered products shown as recommendations

Step 6:

User selects products → added to portfolio

Step 7:

System calculates:

Total Investment
Weighted Return
Risk Distribution

Step 8:

System displays:

- Portfolio summary
- Risk warning (if needed)

8. Conclusion

This project demonstrates how real-world financial systems can be modeled using React. It goes beyond UI development and focuses on:

- Financial reasoning
- Decision-making logic
- Data transformation
- State management